

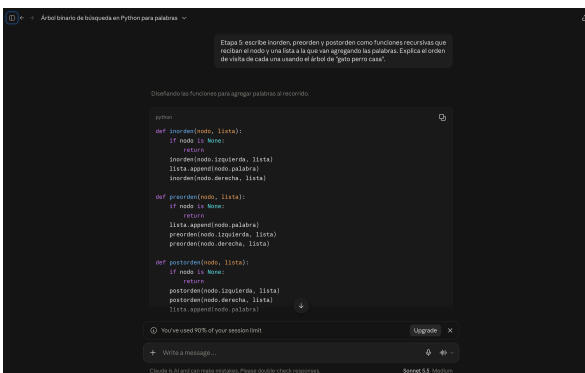
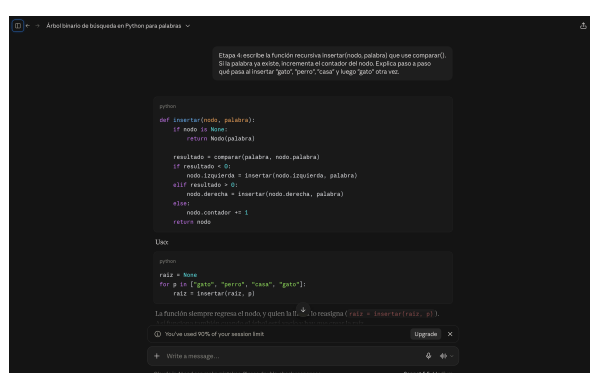
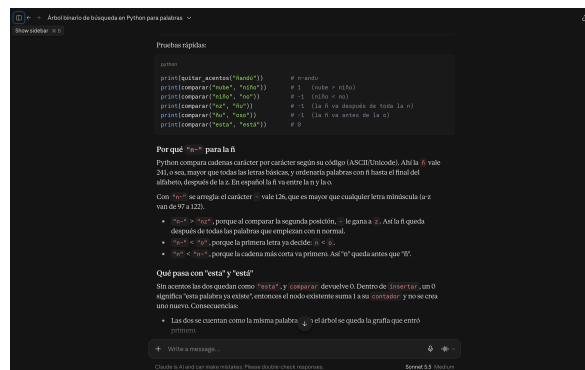
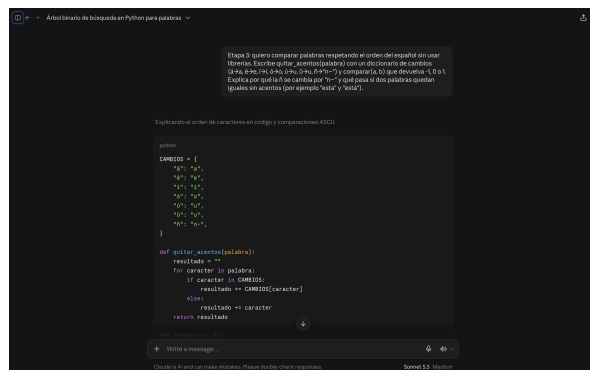
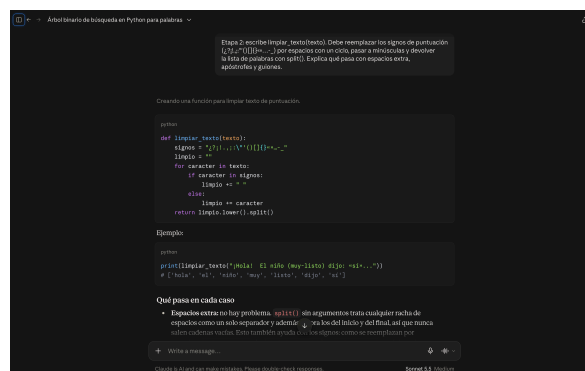
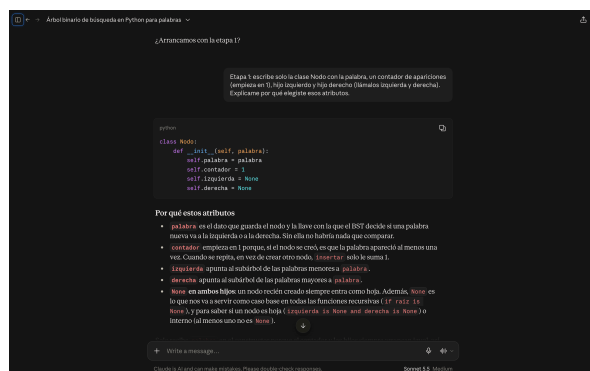
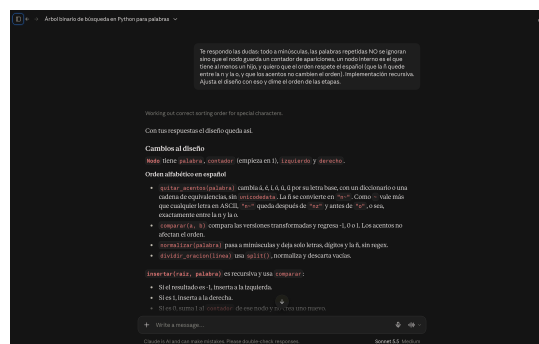
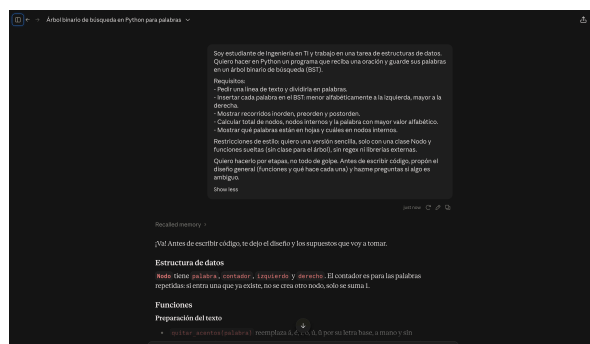
Árboles Binarios

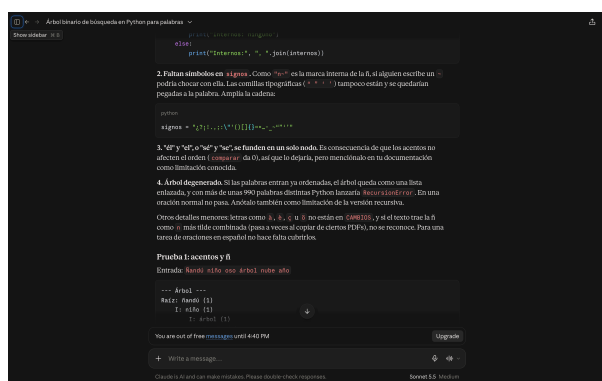
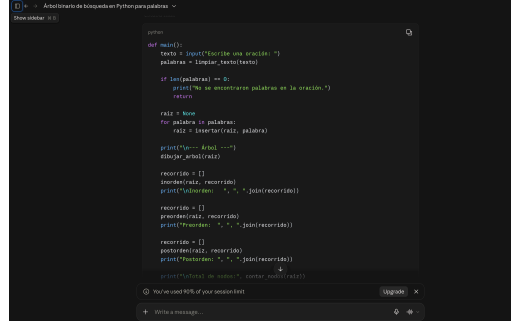
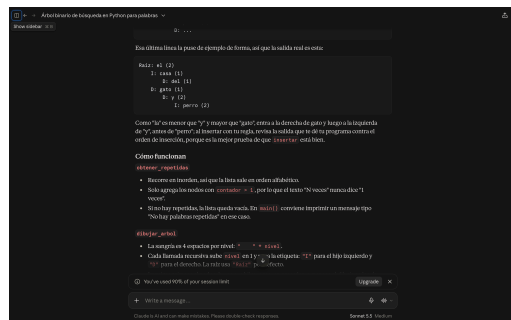
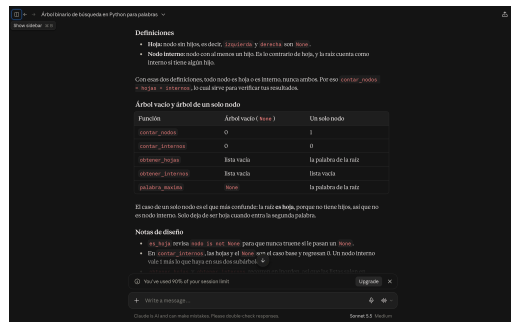
El programa que realizamos con apoyo de la IA recibe una oración, guarda las palabras en árbol binario de búsqueda y muestra los recorridos, conteos y las hojas internas. Fue realizado en python.

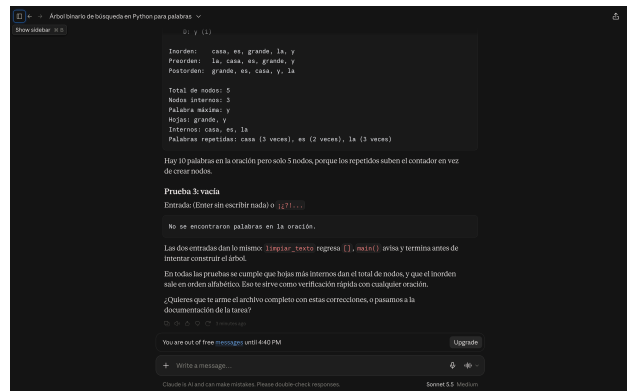
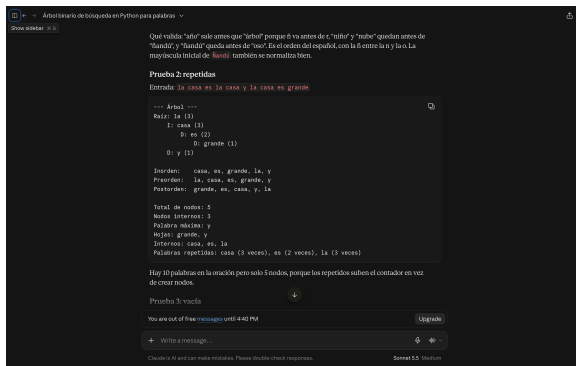
Tabla de Decisiones

Decisión	Qué hace tu código	Justificación
Duplicados	No crea nodo; suma 1 a contador	Permite ver la frecuencia de cada palabra
Mayúsculas	.lower() en limpiar_texto	"Gato" y "gato" son la misma palabra
Puntuación	Reemplaza ¿?!.,;:"'{}«»...- _ por espacio	Los signos no son parte de la palabra
Apóstrofes y guiones	También se reemplazan (don ' t pasa a don y t)	Simplicidad; es una limitación conocida
Acentos	comparar usa la versión sin acentos	Sin esto, "árbol" quedaba después de "zorro"
Ñ	Se convierte en n~ solo para comparar	~ va después de las letras, así que la ñ queda entre n y o
Mismas letras, distinto acento	Desempata con la palabra original (papa / papá)	Son dos nodos distintos, fiel al texto
Nodo interno / hoja	Interno: al menos un hijo. Hoja: sin hijos	Definición estándar
Palabra máxima	Bajar siempre por la derecha	En un BST el mayor es el más a la derecha
Total de nodos	Palabras únicas	Un nodo por palabra distinta

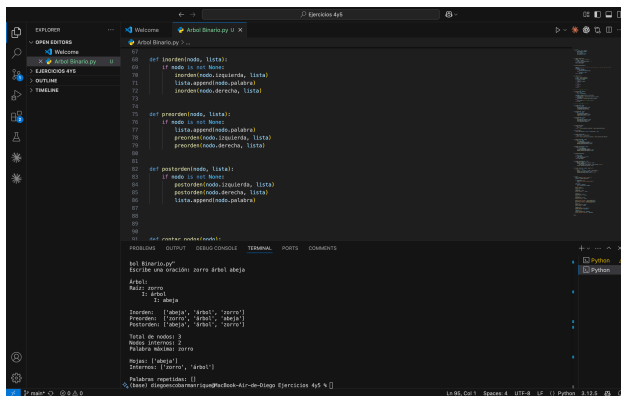
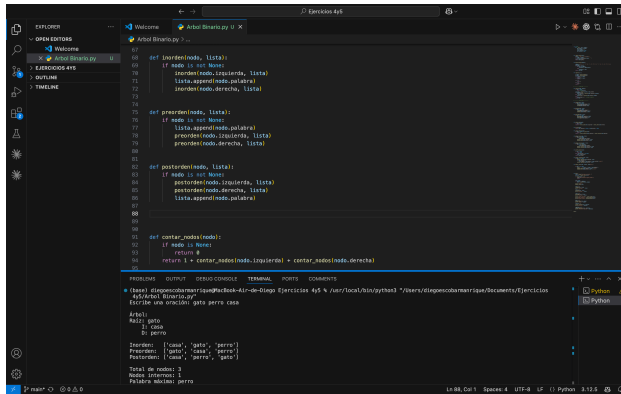
Prompts







Casos de Prueba:




```
def buscar_nodo(nodo, lista):
    if nodo is not None:
        insertar_nodo_izquierda(lista, nodo)
        lista.agregar_nodo(nodo.palabra)
        buscar_nodo(nodo.derecha, lista)

def preordenar_nodo(nodo, lista):
    if nodo is not None:
        lista.agregar_nodo(nodo.palabra)
        preordenar_nodo(nodo.izquierda, lista)
        preordenar_nodo(nodo.derecha, lista)

def postordenar_nodo(nodo, lista):
    if nodo is not None:
        postordenar_nodo(nodo.izquierda, lista)
        postordenar_nodo(nodo.derecha, lista)
        lista.agregar_nodo(nodo.palabra)

# Ejercicio 4: Implementar un algoritmo para contar el número de nodos en un árbol binario.
def contar_nodos(nodo):
    if nodo is None:
        return 0
    else:
        return 1 + contar_nodos(nodo.izquierda) + contar_nodos(nodo.derecha)

# Ejercicio 5: Implementar un algoritmo para encontrar el camino más largo en un árbol binario.
def encontrar_camino_mas_largo(nodo):
    if nodo is None:
        return []
    else:
        izquierda = encontrar_camino_mas_largo(nodo.izquierda)
        derecha = encontrar_camino_mas_largo(nodo.derecha)
        if len(izquierda) > len(derecha):
            return [nodo.palabra] + izquierda
        else:
            return [nodo.palabra] + derecha

# Ejercicio 6: Implementar un algoritmo para encontrar el camino más corto en un árbol binario.
def encontrar_camino_mas_corto(nodo):
    if nodo is None:
        return []
    else:
        izquierda = encontrar_camino_mas_corto(nodo.izquierda)
        derecha = encontrar_camino_mas_corto(nodo.derecha)
        if len(izquierda) < len(derecha):
            return [nodo.palabra] + izquierda
        else:
            return [nodo.palabra] + derecha
```

```
# Ejercicio 7: Implementar un algoritmo para encontrar el camino más largo en un árbol binario.
def encontrar_camino_mas_largo(nodo):
    if nodo is None:
        return []
    else:
        izquierda = encontrar_camino_mas_largo(nodo.izquierda)
        derecha = encontrar_camino_mas_largo(nodo.derecha)
        if len(izquierda) > len(derecha):
            return [nodo.palabra] + izquierda
        else:
            return [nodo.palabra] + derecha

# Ejercicio 8: Implementar un algoritmo para encontrar el camino más corto en un árbol binario.
def encontrar_camino_mas_corto(nodo):
    if nodo is None:
        return []
    else:
        izquierda = encontrar_camino_mas_corto(nodo.izquierda)
        derecha = encontrar_camino_mas_corto(nodo.derecha)
        if len(izquierda) < len(derecha):
            return [nodo.palabra] + izquierda
        else:
            return [nodo.palabra] + derecha

# Ejercicio 9: Implementar un algoritmo para encontrar el camino más largo en un árbol binario.
def encontrar_camino_mas_largo(nodo):
    if nodo is None:
        return []
    else:
        izquierda = encontrar_camino_mas_largo(nodo.izquierda)
        derecha = encontrar_camino_mas_largo(nodo.derecha)
        if len(izquierda) > len(derecha):
            return [nodo.palabra] + izquierda
        else:
            return [nodo.palabra] + derecha

# Ejercicio 10: Implementar un algoritmo para encontrar el camino más corto en un árbol binario.
def encontrar_camino_mas_corto(nodo):
    if nodo is None:
        return []
    else:
        izquierda = encontrar_camino_mas_corto(nodo.izquierda)
        derecha = encontrar_camino_mas_corto(nodo.derecha)
        if len(izquierda) < len(derecha):
            return [nodo.palabra] + izquierda
        else:
            return [nodo.palabra] + derecha
```

```
# Ejercicio 11: Implementar un algoritmo para encontrar el camino más largo en un árbol binario.
def encontrar_camino_mas_largo(nodo):
    if nodo is None:
        return []
    else:
        izquierda = encontrar_camino_mas_largo(nodo.izquierda)
        derecha = encontrar_camino_mas_largo(nodo.derecha)
        if len(izquierda) > len(derecha):
            return [nodo.palabra] + izquierda
        else:
            return [nodo.palabra] + derecha

# Ejercicio 12: Implementar un algoritmo para encontrar el camino más corto en un árbol binario.
def encontrar_camino_mas_corto(nodo):
    if nodo is None:
        return []
    else:
        izquierda = encontrar_camino_mas_corto(nodo.izquierda)
        derecha = encontrar_camino_mas_corto(nodo.derecha)
        if len(izquierda) < len(derecha):
            return [nodo.palabra] + izquierda
        else:
            return [nodo.palabra] + derecha

# Ejercicio 13: Implementar un algoritmo para encontrar el camino más largo en un árbol binario.
def encontrar_camino_mas_largo(nodo):
    if nodo is None:
        return []
    else:
        izquierda = encontrar_camino_mas_largo(nodo.izquierda)
        derecha = encontrar_camino_mas_largo(nodo.derecha)
        if len(izquierda) > len(derecha):
            return [nodo.palabra] + izquierda
        else:
            return [nodo.palabra] + derecha

# Ejercicio 14: Implementar un algoritmo para encontrar el camino más corto en un árbol binario.
def encontrar_camino_mas_corto(nodo):
    if nodo is None:
        return []
    else:
        izquierda = encontrar_camino_mas_corto(nodo.izquierda)
        derecha = encontrar_camino_mas_corto(nodo.derecha)
        if len(izquierda) < len(derecha):
            return [nodo.palabra] + izquierda
        else:
            return [nodo.palabra] + derecha
```

Reflexiones

Rodrigo:

La IA me ayudó con la estructura del árbol: el nodo, la inserción y los tres recorridos. También me propuso casos de prueba que no había pensado, como el texto vacío o las palabras con acento.

Al probar zorro árbol abeja vi que Python ponía "árbol" después de "zorro". Por eso agregamos una función para quitar los acentos antes de comparar. También revisé que las mayúsculas y los signos no crearan nodos repetidos.

Aprendí cómo funciona un árbol binario de búsqueda y que la IA ayuda, pero no siempre acierta. Hay que probar y entender el código antes de usarlo.

Freddy:

Usé la IA para construir por etapas un programa en Python que guarda las palabras de una oración en un árbol binario de búsqueda, y así entendí cómo funcionan la inserción recursiva, los recorridos y el orden alfabético en español. Probé cada etapa y comparé versiones para poder explicar el código con mis propias palabras, no solo copiarlo.

Diego:

La IA me permitió darle una estructura y orden al código, corregir funciones que estaban mal hechas y comprender el uso de árbol de búsqueda binario, y la utilidad que se le puede dar, de igual manera entender mejor a como programar de manera ordenada y en conjunto con mis compañeros, sobre todo que la IA es una herramienta muy buena para aprender y no es solo para copiar y pegar.